

Student Droupout System

Problem Statement

Students often need to be divided into groups for classroom projects, assignments, and activities. Creating these groups manually takes time and may lead to unbalanced teams. The **Student Droupout System** automates this process by generating fair and balanced groups using student data.

Objectives

- ✓ Automate student group formation.
 - ✓ Reduce manual effort.
 - ✓ Create balanced groups.
 - ✓ Improve teamwork and collaboration.
-

Stakeholders

-  Students
 -  Teachers
 -  College Administration
-

AI Task

The AI analyzes student information and automatically creates balanced groups based on predefined criteria such as department, skills, or group size.

Can This Problem Be Solved Without AI?

Yes. Groups can be created manually, but AI makes the process faster, more accurate, and scalable for large numbers of students.

Constraints

- Student data should be accurate.
- Group sizes should remain balanced.
- Missing values may affect the output.

Assumptions

- Student records are complete.
- Every student is assigned to one group only.
- The dataset is clean and well-formatted.

Expected Output

The system automatically generates fair and balanced student groups, helping teachers save time while improving collaboration among students.

Let's Start Coding!

```
In [1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

Step 1: Generate Student Dataset

Purpose

In this step, we create a **synthetic student dataset** using **NumPy** and **Pandas**. This dataset simulates real-world student records and will be used for training and testing our Machine Learning model.

Dataset Features

 Feature	 Description
 Age	Student's age (17–25 years)
 Gender	Male or Female
 Attendance	Attendance percentage (40–100%)
 CGPA	Academic performance
 Backlogs	Number of pending subjects
 Family Income	Annual family income
 Study Hours	Daily study hours
 Scholarship	Scholarship status (0 = No, 1 = Yes)
 Internet Access	Internet availability (0 = No, 1 = Yes)
 Hostel	Hostel resident (0 = No, 1 = Yes)
 Mental Stress	Stress level (1–10)
 Part-Time Job	Part-time job status (0 = No, 1 = Yes)

What This Code Does

- ✓ Sets a random seed for reproducibility.
- ✓ Generates **10,000 student records**.
- ✓ Creates multiple student-related attributes.
- ✓ Stores all records in a **Pandas DataFrame** named **df**.

Expected Output

A structured dataset containing **10,000 student records** with demographic, academic, financial, and personal information, ready for preprocessing and

```
In [2]: np.random.seed(42)
n = 10000

df = pd.DataFrame({
    "Age": np.random.randint(17,26, n),
    "Gender": np.random.choice(["Male","Female"], n),
    "Attendance": np.random.randint(40, 101, n),
    "CGPA": np.round(np.random.uniform(4.5, 10, n),2),
    "Backlogs": np.random.randint(0,8,n),
    "FamilyIncome": np.random.randint(150000, 12000000, n),
    "StudyHours": np.round(np.random.uniform(0.5,8,n),1),
    "Scholarship": np.random.choice([0,1],n, p=[0.7,0.3]),
    "InternetAccess": np.random.choice([0,1],n, p=[0.15,0.85]),
    "Hostel": np.random.choice([0,1],n),
    "MentalStress": np.random.randint(1,11,n),
    "PartTimeJob": np.random.choice([0,1],n, p=[0.75,0.25])
})
```

🕒 Step 2: Display the Dataset

🎯 Purpose

This step displays the generated **Student Dataset** in a tabular format. It allows us to verify that the dataset has been created successfully and to inspect the values of each feature.

📋 What This Step Does

- ✓ Displays all student records stored in the DataFrame.
- ✓ Shows every feature (column) and its corresponding values.
- ✓ Helps verify that the data generation process was successful.
- ✓ Provides an overview of the dataset before preprocessing.

📊 Expected Output

A table containing **10,000 student records** with features such as:

-  Age
-  Gender
-  Attendance
-  CGPA
-  Backlogs
-  Family Income
-  Study Hours
-  Scholarship
-  Internet Access
-  Hostel
-  Mental Stress
-  Part-Time Job

 **Observation:** The dataset appears in a structured table where each row represents one student and each column represents a specific student attribute.

In [3]: `df`

Out[3]:

	Age	Gender	Attendance	CGPA	Backlogs	FamilyIncome	StudyHours	Scho
0	23	Female	70	5.71	3	2744921	2.5	
1	20	Female	45	6.17	6	8791810	1.2	
2	24	Male	53	5.21	7	2874959	7.9	
3	21	Female	97	5.87	0	10029209	6.2	
4	23	Male	62	9.02	2	646632	0.5	
...	...	...	...	...	...	...	...	...
9995	17	Female	70	4.77	6	2549318	7.5	
9996	23	Male	48	7.05	5	7781440	5.5	
9997	22	Male	61	9.10	0	7274217	2.8	
9998	25	Female	50	5.49	5	9772777	5.2	
9999	23	Female	56	5.30	5	10765720	5.0	

10000 rows × 12 columns

Step 3: Create the Target Variable (Dropout)

 Purpose

In this step, we create the **target variable** named **Dropout**. The target variable represents whether a student is likely to **drop out (1)** or **continue their studies (0)** based on multiple academic and personal risk factors.

Risk Factors Used

 Risk Factor	 Condition
 Attendance	Less than 65%
 CGPA	Less than 6.0
 Backlogs	3 or more
 Mental Stress	8 or above
 Study Hours	Less than 2 hours/day

Logic Used

- ✓ Each risk condition contributes **1 point**.
- ✓ The total risk score is calculated for every student.
- ✓ If the **risk score is 3 or more**, the student is labeled as:
 - ● **1 = Likely to Drop Out**
 - ● **0 = Not Likely to Drop Out**

Expected Output

A new column named **Dropout** is added to the dataset. This column serves as the **target variable** for training the Machine Learning model.

 **Note:** Since this is a synthetic dataset, the **Dropout** labels are generated using predefined rules instead of real student outcomes.

```
In [4]: # Low attendance | Low CGPA | Many Backlogs | High Stress
risk = (
    (df["Attendance"] < 65).astype(int) +
    (df["CGPA"] < 6.0).astype(int) +
    (df["Backlogs"] >= 3).astype(int) +
    (df["MentalStress"] >= 8).astype(int) +
```

```
(df["StudyHours"]< 2).astype(int)
)
df["Dropout"] = np.where(risk>= 3,1,0)
```

Step 4: Preview the Dataset

Purpose

This step displays the **first five rows** of the dataset to verify that the data has been generated correctly and that the **Dropout** column has been added successfully.

Why Use `df.head()` ?

- ✓ Displays the first **5 student records**.
- ✓ Confirms that all features are present.
- ✓ Verifies that the **Dropout** target column has been created.
- ✓ Allows a quick inspection of the dataset before preprocessing and model training.

Expected Output

A table showing the **first five rows** of the Student Group System dataset, including features such as:

-  Age
-  Gender
-  Attendance
-  CGPA
-  Backlogs
-  Family Income
-  Study Hours
-  Scholarship
-  Internet Access
-  Hostel
-  Mental Stress

-  Part-Time Job
-  Dropout

 **Observation:** Review the displayed records to ensure the data looks correct before moving on to data preprocessing and machine learning.

In [5]: `df.head()`

Out[5]:

	Age	Gender	Attendance	CGPA	Backlogs	FamilyIncome	StudyHours	Scholars
0	23	Female	70	5.71	3	2744921	2.5	
1	20	Female	45	6.17	6	8791810	1.2	
2	24	Male	53	5.21	7	2874959	7.9	
3	21	Female	97	5.87	0	10029209	6.2	
4	23	Male	62	9.02	2	646632	0.5	

Step 5: Separate Features and Target Variable

Purpose

In this step, we divide the dataset into **input features (X)** and the **target variable (y)**. This is an essential step before training a Machine Learning model.

What are Features (X)?

The **features (X)** contain all the student information that will be used to predict whether a student is likely to drop out.

These include:

-  Age
-  Gender
-  Attendance
-  CGPA
-  Backlogs
-  Family Income

- 🕒 Study Hours
- 🎓 Scholarship
- 🌐 Internet Access
- 🏠 Hostel
- 😞 Mental Stress
- 💼 Part-Time Job

Target Variable (y)

The **target variable (y)** is the **Dropout** column.

- ● 0 → Student is **Not Likely to Drop Out**
- ● 1 → Student is **Likely to Drop Out**

This is the value that our Machine Learning model will learn to predict.

What This Code Does

- ✓ Removes the **Dropout** column from the dataset and stores the remaining columns in **X**.
- ✓ Stores the **Dropout** column separately in **y**.
- ✓ Displays **X** to verify that it contains only the input features.

Expected Output

A DataFrame containing all **student features** except the **Dropout** column. This feature set will be used to train and evaluate the Machine Learning model.

💡 **Note:** Separating features (**X**) and the target (**y**) is a standard preprocessing step before splitting the data into training and testing sets.

```
In [6]: X = df.drop("Dropout",axis=1)
        y = df["Dropout"]
        X
```

Out[6]:

	Age	Gender	Attendance	CGPA	Backlogs	FamilyIncome	StudyHours	Scho
0	23	Female	70	5.71	3	2744921	2.5	
1	20	Female	45	6.17	6	8791810	1.2	
2	24	Male	53	5.21	7	2874959	7.9	
3	21	Female	97	5.87	0	10029209	6.2	
4	23	Male	62	9.02	2	646632	0.5	
...	...	...	...	...	...	...	...	...
9995	17	Female	70	4.77	6	2549318	7.5	
9996	23	Male	48	7.05	5	7781440	5.5	
9997	22	Male	61	9.10	0	7274217	2.8	
9998	25	Female	50	5.49	5	9772777	5.2	
9999	23	Female	56	5.30	5	10765720	5.0	

10000 rows × 12 columns

↻ Step 6: Convert Categorical Data into Numerical Data

🎯 Purpose

Machine Learning algorithms can only work with **numerical data**. In this step, we convert categorical features (such as **Gender**) into numerical values using **One-Hot Encoding**.

🔑 Why is Encoding Required?

Some columns contain **text values**, for example:

- 🧑 Gender = Male / Female

Since Machine Learning models cannot directly process text data, these values must be converted into numbers.

⚙️ What This Code Does

- ✓ Identifies all categorical columns.
- ✓ Converts them into binary (0 or 1) columns using **One-Hot Encoding**.
- ✓ Uses `drop_first=True` to remove one category and avoid redundant information (also known as the dummy variable trap).

Example

Before Encoding	After Encoding
Male	1
Female	0

Expected Output

The feature dataset **X** will contain only **numerical values**, making it ready for Machine Learning model training.

 **Note:** After this step, all features are in a format that Machine Learning algorithms can process efficiently.

```
In [7]: # y
X = pd.get_dummies(X, drop_first=True)
```

Step 7: Display the Processed Feature Dataset

Purpose

This step displays the **processed feature dataset (X)** after encoding. It helps verify that all categorical variables have been successfully converted into numerical values.

What This Step Does

- ✓ Displays all input features used for training the Machine Learning model.
- ✓ Confirms that categorical columns (such as **Gender**) have been encoded.
- ✓ Ensures the dataset now contains only numerical values.
- ✓ Verifies that the data is ready for the train-test split.

Expected Output

A DataFrame containing all processed input features, where every column is numerical and suitable for Machine Learning algorithms.

 **Observation:** Check that there are no text-based columns remaining. The dataset should now be fully prepared for model training.

In [8]: X

Out[8]:

	Age	Attendance	CGPA	Backlogs	FamilyIncome	StudyHours	Scholarship
0	23	70	5.71	3	2744921	2.5	0
1	20	45	6.17	6	8791810	1.2	0
2	24	53	5.21	7	2874959	7.9	0
3	21	97	5.87	0	10029209	6.2	0
4	23	62	9.02	2	646632	0.5	0
...	...	...	...	...	...	...	...
9995	17	70	4.77	6	2549318	7.5	1
9996	23	48	7.05	5	7781440	5.5	0
9997	22	61	9.10	0	7274217	2.8	1
9998	25	50	5.49	5	9772777	5.2	1
9999	23	56	5.30	5	10765720	5.0	1

10000 rows × 12 columns

Step 8: Split the Dataset into Training and Testing Sets

Purpose

Before training a Machine Learning model, the dataset is divided into **training** and **testing** sets. This allows us to train the model on one portion of the data and evaluate its performance on unseen data.

Dataset Split

 Dataset	 Percentage	 Purpose
● Training Set	80%	Used to train the Machine Learning model
● Testing Set	20%	Used to evaluate the model's performance

What This Code Does

- ✓ Splits the feature dataset (**X**) and target variable (**y**) into training and testing sets.
- ✓ Uses **80%** of the data for training.
- ✓ Uses **20%** of the data for testing.
- ✓ Sets **random_state = 41** to ensure the same split is produced every time the code runs.
- ✓ Uses **stratify = y** to maintain the same proportion of dropout and non-dropout students in both the training and testing datasets.

Output Variables

Variable	Description
X_train	Training features
X_test	Testing features
y_train	Training target values
y_test	Testing target values

Expected Output

The dataset is successfully divided into **training** and **testing** sets, making it ready for Machine Learning model training and performance evaluation.

 **Note:** Separating training and testing data helps evaluate how well the model performs on new, unseen student records.

```
In [9]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size = 0.2,
        random_state = 41,
        stratify=y
    )
```

Step 9: Check the Shape of the Training Dataset

Purpose

This step checks the **dimensions (shape)** of the training feature dataset. It helps verify that the data has been split correctly and is ready for training the Machine Learning model.

What Does `.shape` Show?

The `.shape` attribute returns two values:

-  **Rows** → Number of training samples (students).
-  **Columns** → Number of input features used for prediction.

Why is This Important?

- ✓ Confirms that the training dataset contains the expected number of records.
- ✓ Verifies that all input features are present.

✓ Ensures the data is correctly prepared before model training.

Expected Output

The output will appear in the format:

(Number of Training Records, Number of Features)

For example:

```
(8000, 12)
```

This means:

- **8000** student records are available for training.
- **12** input features are used by the Machine Learning model.

 **Observation:** Since the dataset was split into **80% training** and **20% testing**, the training set should contain approximately **8,000 records** out of **10,000**.

Step 9: Check the Shape of the Training Dataset

Objective

The purpose of this step is to determine the **size and dimensions** of the training dataset after splitting the original data.

What Does `X_train.shape` Mean?

The `.shape` function returns a tuple containing:

-  **Number of Rows** → Total training samples (students).
-  **Number of Columns** → Total input features used for model training.

Why Is This Important?

- ✓ Verifies that the dataset has been split correctly.
- ✓ Confirms the number of student records used for training.
- ✓ Ensures all required features are available before training the Machine Learning model.

Expected Output

For example:

(8000, 12)

Interpretation:

-  **8000** → Number of students in the training dataset.
-  **12** → Number of input features available for prediction.

 **Note:** Since **80%** of the dataset is used for training, the training dataset should contain approximately **8,000 student records** out of **10,000**.

```
In [10]: X_train.shape
```

```
Out[10]: (8000, 12)
```

```
In [11]: X_test.shape
```

```
Out[11]: (2000, 12)
```

Step 10: Check the Shape of the Training Target Variable

Objective

This step checks the **size of the training target variable (`y_train`)**. It confirms that every training record has a corresponding target label for model training.

 **What Does `y_train.shape` Show?**

The `.shape` function returns the **number of target labels** in the training dataset.

Each value in `y_train` represents:

-  **0** → Student is **Not Likely to Drop Out**
-  **1** → Student is **Likely to Drop Out**

Why Is This Important?

- ✓ Verifies that the target variable has been split correctly.
- ✓ Ensures every training sample has a corresponding target label.
- ✓ Confirms that the data is ready for Machine Learning model training.

Expected Output

For example:

`(8000,)`

Interpretation:

-  **8000** → Number of target labels in the training dataset.
-  The single value indicates that `y_train` is a **one-dimensional array (Series)** containing one dropout label for each student.

 **Note:** The number of rows in `y_train` should always be the same as the number of rows in `X_train`, ensuring that each student record has a corresponding target value.

```
In [12]: y_train.shape
```

```
Out[12]: (8000,)
```

Step 11: Check the Shape of the Testing Target Variable

 Objective

This step checks the **size of the testing target variable (`y_test`)**. It ensures that the testing dataset contains the correct number of target labels for evaluating the Machine Learning model.

What Does `y_test.shape` Show?

The `.shape` function returns the **number of target labels** in the testing dataset.

Each value in `y_test` represents:

-  **0** → Student is **Not Likely to Drop Out**
-  **1** → Student is **Likely to Drop Out**

Why Is This Important?

- ✓ Verifies that the testing target data has been split correctly.
- ✓ Ensures every testing record has a corresponding target label.
- ✓ Confirms that the testing dataset is ready for evaluating the Machine Learning model.

Expected Output

For example:

`(2000, )`

Interpretation:

-  **2000** → Number of target labels in the testing dataset.
-  The single value indicates that `y_test` is a **one-dimensional array (Series)** containing one dropout label for each student.

 **Note:** Since **20%** of the dataset is reserved for testing, `y_test` should contain approximately **2,000 target values** (out of a total of 10,000 records), matching the number of rows in `X_test`.

```
In [13]: y_test.shape
```

Step 12: Import Machine Learning Algorithms

Purpose

In this step, we import various **Machine Learning classification algorithms**. These models will be trained and compared to determine which one predicts student dropout most accurately.

Algorithms Used

 Algorithm	 Purpose
 Logistic Regression	Predicts dropout using a linear classification approach.
 K-Nearest Neighbors (KNN)	Classifies students based on the similarity of nearby data points.
 Decision Tree	Makes predictions using a tree-like structure of decision rules.
 Random Forest	Combines multiple decision trees to improve prediction accuracy.
 Support Vector Machine (SVM)	Finds the best boundary to separate dropout and non-dropout students.
 XGBoost	An advanced boosting algorithm known for high performance and accuracy.
 Gaussian Naive Bayes	Uses probability to classify students based on feature distributions.

Why Import Multiple Models?

- ✓ Compare the performance of different algorithms.
- ✓ Identify the model with the highest prediction accuracy.
- ✓ Evaluate which algorithm works best for the Student Group System dataset.

Expected Outcome

All required Machine Learning models are successfully imported and ready to be trained using the training dataset.

 **Note:** Instead of relying on a single algorithm, testing multiple models helps select the one that provides the best performance for predicting student dropout.

```
In [22]: from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
```

Step 13: Initialize Machine Learning Models

Purpose

In this step, we create a **dictionary of Machine Learning models**. Each model is assigned a name and initialized with its default (or specified) parameters. This allows us to train and compare multiple models efficiently.

Models Included

 Model	 Description
 Logistic Regression	A linear classification algorithm used for binary prediction.
 K-Nearest Neighbors (KNN)	Classifies students based on the nearest neighboring data points.
 Decision Tree	Uses a tree-like structure to make classification decisions.
 Random Forest	Combines multiple decision trees to improve prediction accuracy.
 Support Vector Machine (SVM)	Separates classes by finding the optimal decision boundary.
 XGBoost	A powerful gradient boosting algorithm for high-performance classification.

 Gaussian Naive Bayes	Uses probability theory to classify student records.
--	--

What This Code Does

- ✓ Creates a Python dictionary named `models`.
- ✓ Stores each model with a descriptive name.
- ✓ Sets `max_iter=1000` for Logistic Regression to ensure the model has enough iterations to converge.
- ✓ Makes it easy to train and evaluate all models using a loop.

Expected Output

The `models` dictionary contains all the initialized Machine Learning algorithms, ready for training and performance comparison on the Student Group System dataset.

 **Note:** Using a dictionary allows us to write cleaner code by training multiple models in a loop instead of writing separate code for each algorithm.

```
In [23]: models = {  
    "Logistic Regression": LogisticRegression(max_iter=1000),  
    "KNN": KNeighborsClassifier(),  
    "Decision Tree": DecisionTreeClassifier(),  
    "Random Forest": RandomForestClassifier(),  
    "SVM": SVC(),  
    "XGBoost": XGBClassifier(),  
    "Naive Bayes": GaussianNB()  
}
```

Step 14: Train the Machine Learning Models and Make Predictions

Purpose

In this step, each Machine Learning model is trained using the **training dataset** and then used to predict whether students in the **testing dataset** are likely to drop out.

How the Process Works

- 1 Select one model from the **models** dictionary.
- 2 Train the model using **X_train** and **y_train**.
- 3 Use the trained model to make predictions on **X_test**.
- 4 Repeat the same process for all Machine Learning models.

What This Code Does

 Code	 Description
<code>for name, model in models.items()</code>	Loops through each Machine Learning model.
<code>model.fit(X_train, y_train)</code>	Trains the model using the training dataset.
<code>model.predict(X_test)</code>	Predicts the dropout status for the testing dataset.
<code>result = []</code>	Creates an empty list to store the evaluation results of each model.

Expected Output

Each Machine Learning model is successfully trained and generates predictions for the testing dataset. These predictions will be used in the next step to calculate evaluation metrics such as **Accuracy, Precision, Recall, and F1-Score**.

 **Note:** Training multiple models in a loop saves time, reduces repetitive code, and makes it easier to compare the performance of different algorithms.

```
In [24]: result = []
         for name, model in models.items():
```

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

Step 15: Train Models and Evaluate Their Performance

Purpose

In this step, each Machine Learning model is trained using the training dataset and then evaluated on the testing dataset. The evaluation helps determine which model performs best for predicting student dropout.

Evaluation Metrics Imported

 Metric	 Description
 Accuracy	Measures the overall percentage of correct predictions.
 Precision	Indicates how many predicted dropouts are actually dropouts.
 Recall	Measures how many actual dropouts are correctly identified.
 F1-Score	Balances Precision and Recall into a single score.
 ROC-AUC Score	Measures the model's ability to distinguish between dropout and non-dropout students.

What This Code Does

- ✓ Imports the required evaluation metrics.
- ✓ Trains each Machine Learning model using the training dataset.
- ✓ Predicts dropout labels for the testing dataset.
- ✓ Calculates prediction probabilities when supported by the model.
- ✓ Stores the model name and evaluation results in the **results** list for comparison.

Model Evaluation Workflow

- ① Train the model using `X_train` and `y_train`.
 - ② Predict the dropout status using `X_test`.
 - ③ Generate prediction probabilities (if supported).
 - ④ Calculate performance metrics.
 - ⑤ Save the results for comparison.
-

Expected Output

A list named `results` containing the performance of each Machine Learning model. These results can later be converted into a table to compare which algorithm performs best.

 **Note:** Comparing multiple evaluation metrics provides a more reliable assessment than relying on accuracy alone, especially when working with classification problems.

```
In [25]: from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

results = []

for name, model in models.items():

    # Train
    model.fit(X_train, y_train)

    # Predict class labels
    y_pred = model.predict(X_test)

    # Predict probabilities
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
    else:
        # SVM without probability=True
        y_prob = y_pred

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1 Score": f1_score(y_test, y_pred),
```



```
"ROC AUC": roc_auc_score(y_test, y_prob)
})
```

```
C:\Users\ASharma\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

```
C:\Users\ASharma\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```



Step 16: Create and Display the Model Comparison Table

Purpose

In this step, the evaluation results of all Machine Learning models are converted into a **Pandas DataFrame**. The models are then sorted by **Accuracy** to identify the best-performing algorithm.



What This Code Does

- ✓ Converts the **results** list into a **Pandas DataFrame**.
- ✓ Sorts the models in **descending order of Accuracy**.
- ✓ Displays the comparison table.
- ✓ Makes it easy to compare the performance of all Machine Learning models.



Comparison Metrics

The table may include the following evaluation metrics:

Metric	Description
Accuracy	Overall prediction accuracy
✓ Precision	Correctly predicted dropout cases

 Metric	 Description
 Recall	Correctly identified actual dropout cases
 F1-Score	Balance between Precision and Recall
 ROC-AUC	Model's ability to distinguish between classes

Expected Output

A neatly organized table where:

-  Each row represents a Machine Learning model.
-  Each column shows a performance metric.
-  The **best-performing model** appears at the top because the table is sorted by **Accuracy** in descending order.

 **Observation:** The first row of the table represents the model with the **highest accuracy**, making it the strongest candidate for predicting student dropout in this project.

```
In [26]: result_df = pd.DataFrame(results)
result_df = result_df.sort_values(
    by="Accuracy",
    ascending = False
)
result_df
```

```
Out[26]:
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
3	Random Forest	1.0000	1.000000	1.000000	1.000000	1.000000
5	XGBoost	1.0000	1.000000	1.000000	1.000000	1.000000
2	Decision Tree	0.9990	0.997908	0.997908	0.997908	0.998625
0	Logistic Regression	0.8605	0.769648	0.594142	0.670602	0.906340
4	SVM	0.7610	0.000000	0.000000	0.000000	0.500000
6	Naive Bayes	0.7610	0.000000	0.000000	0.000000	0.647090
1	KNN	0.7015	0.203980	0.085774	0.120766	0.486826

```
In [ ]:
```